

Projeto em Arquiteturas de Sistemas de Informação Distribuídos

Mestrado em Engenharia e Gestão de Sistemas de Informação (ESI)

Mestrado em Engenharia de Ciência de Dados

Ano Letivo 2025/2026

1. Descrição e Objetivos

O objetivo deste projeto é estudar/conceber e avaliar arquiteturas de sistemas de informação distribuídos baseadas em micro-serviços, integrando fundamentação teórica com validação empírica.

O trabalho deverá evidenciar decisões arquiteturais explícitas, análise de alternativas, discussão de *trade-offs* e avaliação com base em atributos de qualidade, nomeadamente *performance*, *availability*, *deployability* e *cost*.

Os recursos do *Google Cloud for Education* poderão ser utilizados como infraestrutura experimental, não constituindo o foco principal do projeto.

O projeto será desenvolvido em grupos de 4 a 5 estudantes.

2. Modalidades de Desenvolvimento

O projeto poderá assumir uma das seguintes modalidades:

- Análise, evolução e avaliação de uma aplicação já baseada em micro-serviços.
- Refactoring* arquitetural de uma aplicação monolítica para uma arquitetura baseada em micro-serviços;

3. Sistemas de Referência (sugestões)

3.1 Análise, evolução e avaliação de uma aplicação já baseada em micro-serviços.

Os sistemas seguintes são exemplos públicos frequentemente usados em investigação/ensino. O “Tamanho” é indicado de forma aproximada (pode variar consoante a definição de micro-serviço e se se contam componentes de infraestruturas como bases de dados, *message brokers*, etc.).

Sistema	Tipo	Tamanho (aprox.)	Repositório
Sock Shop (Weaveworks/ <i>microservices-demo</i>)	<i>Microservices demo</i> (e-commerce)	≈13 micro-serviços (incl. componente s de suporte em alguns <i>deployments</i>)	https://github.com/microservices-demo/microservices-demo , ultimo acesso 14/2/2026. Aplicação de e-commerce composta por múltiplos micro-serviços independentes (<i>frontend</i> , catálogo, carrinho, encomendas, utilizadores, pagamentos), comunicando maioritariamente via REST. Utiliza uma arquitetura em <i>containers</i> , com serviços implementados em diferentes linguagens e bases de dados dedicadas.
Online Boutique (GCP <i>microservices-demo</i>)	<i>Microservices demo</i> (e-commerce)	10 micro-serviços	https://github.com/GoogleCloudPlatform/microservices-demo , ultimo acesso 14/2/2026. Aplicação de e-commerce composta por 10 micro-serviços independentes, implementados em várias linguagens (Go, Java, Python, Node.js, C#, entre outras), comunicando predominantemente via gRPC. Concebida para execução em Kubernetes.
DeathStarBench – Hotel Reservation	<i>Benchmark académico</i> (Go + gRPC)	≈18–22 micro-serviços	https://github.com/delimitrou/DeathStarBench/tree/master/hotelReservation , ultimo acesso 14/2/2026. <i>Benchmark</i> académico baseado numa aplicação de reservas de hotel composta por vários micro-serviços que comunicam predominantemente via gRPC. Implementado maioritariamente em Go, inclui serviços de <i>frontend</i> , perfil, reserva, recomendação e gestão de utilizadores, com dependências distribuídas.

Train-Ticket (FudanSELab)	Benchmark acadêmico (multi-linguagem)	≈41 micro-serviços (varia por <i>fork</i> /versão)	https://github.com/FudanSELab/train-ticket/tree/v0.2.0 , ultimo acesso 14/2/2026. Sistema distribuído de grande dimensão que implementa uma plataforma de reservas de comboio. Utiliza várias linguagens (predominantemente Java/Spring Boot), bases de dados por serviço e comunicação REST entre serviços.
---------------------------	---------------------------------------	--	---

3.2 Sugestões para projetos de *Refactoring* (monólito → micro-serviços)

- Online-Book-Store-Spring-Boot: <https://github.com/Janith3cx/Online-Book-Store-Spring-Boot>, ultimo acesso 14/02/2026. Aplicação monolítica de e-commerce desenvolvida em Java com Spring Boot, estruturada segundo arquitetura MVC tradicional. Integra persistência relacional e API REST num único artefacto executável.
- Spring PetClinic (monólito): <https://github.com/spring-projects/spring-petclinic>, ultimo acesso 14/02/2026. Aplicação monolítica em Java baseada em Spring Framework, organizada em camadas (controladores, serviços, persistência) e suportada por base de dados relacional. Implementa um domínio simples de gestão clínica veterinária.
- XCommerce Monolithic (Spring Boot): <https://github.com/oiraqi/xcommerce-monolithic>, ultimo acesso 14/02/2026. Aplicação monolítica de e-commerce em Java/Spring Boot, estruturada segundo um modelo MVC com organização interna orientada a serviços (estilo SOA). Integra JPA/Hibernate, PostgreSQL, Redis, REST e autenticação baseada em JWT num único *deployment*.
- Trébol Backend Monolith (Spring Boot): <https://github.com/trebol-ecommerce/trebol-backend-monolith>, ultimo acesso 14/02/2026. *Backend* monolítico de e-commerce desenvolvido em Spring Boot, expondo API REST estruturada e integrando persistência relacional. Apresenta separação interna por camadas e módulos, mas é executado como um único artefacto aplicacional.

4. Requisitos

4.1 Fundamentação Arquitetural

O projeto deverá evidenciar uma análise arquitetural fundamentada. Deverá incluir:

- Identificação clara da decisão arquitetural analisada ou introduzida;
- Descrição de pelo menos uma alternativa relevante;
- Discussão fundamentada de *trade-offs* entre alternativas;
- Relação explícita da decisão com os atributos de qualidade:
 - o *Performance*
 - o *Availability*
 - o *Deployability*
 - o *Cost*
- Justificação técnica das opções tomadas, com base em literatura ou boas práticas reconhecidas.

A fundamentação arquitetural constitui a componente central do projeto.

4.2 Implementação e *Deployment* (Suporte Experimental)

A implementação deverá servir como meio de validação empírica das decisões arquiteturais. Deverá incluir:

- Implementação mínima necessária para avaliar a hipótese formulada;
- *Deployment* em ambiente cloud (*Google Cloud for Education*) ou equivalente;
- Definição e justificação das configurações de recursos (CPU, memória, número de réplicas, etc.);
- Estratégia de *deployment* coerente com os atributos de qualidade analisados (por exemplo, escalabilidade, tolerância a falhas, facilidade de atualização);

- Mecanismos básicos de monitorização.

A implementação não é um fim em si mesma, mas um instrumento para suportar a análise arquitetural.

4.3 Avaliação Experimental

O projeto deverá incluir um estudo experimental estruturado que permita estabelecer relações entre decisão arquitetural e atributos de qualidade. Deverá incluir:

- Formulação clara dos objetivos;
- Definição de questões a estudar;
- Definição de modelo de carga;
- Repetição de experiências sempre que aplicável;
- Recolha de métricas (exemplos):
 - o Latência média
 - o *Throughput*
 - o Utilização de CPU e memória
 - o Indicadores relevantes de disponibilidade
 - o Estimativa de custo operacional
 - o Análise estatística básica (médias, variação, comparação entre cenários);
- Interpretação crítica dos resultados à luz da decisão arquitetural estudada.

A avaliação deverá demonstrar como os resultados obtidos confirmam, ou não, a decisão arquitetural analisada.

5. Temas Orientadores

Os temas seguintes são indicativos e deverão ser desenvolvidos no contexto de uma das modalidades definidas na Secção 2. Em todos os casos, o trabalho deverá estabelecer uma relação entre decisão arquitetural e os atributos de qualidade *performance*, *availability*, *deployability* e *cost*.

5.1 Análise, Evolução e Avaliação de uma Aplicação Baseada em Micro-serviços

1. Configuração de Recursos e Performance

Analisar o impacto de diferentes configurações de CPU (*requests/limits*) e memória na latência, *throughput* e estabilidade do sistema. Avaliar igualmente o impacto no custo operacional e discutir implicações na disponibilidade sob carga elevada.

2. Escalabilidade Horizontal e Custo Marginal

Estudar a evolução do desempenho com o aumento do número de réplicas. Identificar pontos de saturação e analisar o equilíbrio entre *performance*, *availability* e custo incremental.

3. Estratégias de Comunicação entre Serviços

Analisar a comunicação existente (por exemplo, síncrona via REST) e propor alternativa arquitetural. Avaliar impacto na latência acumulada, disponibilidade em caso de falha parcial e complexidade de deployment.

4. Estratégias de Escalabilidade: Auto-scaling vs Provisionamento Fixo

Comparar *auto-scaling* com provisionamento estático. Analisar efeitos em estabilidade da *performance*, disponibilidade sob carga variável e custo total.

5. Observabilidade e Disponibilidade Operacional

Analisar como a arquitetura influencia a capacidade de monitorização e deteção de falhas. Relacionar decisões arquiteturais com *availability* e custo operacional associado à gestão do sistema.

5.2 Refactoring Arquitetural de uma Aplicação Monolítica

1. Estratégia de Decomposição e Performance

Propor uma decomposição da aplicação monolítica em micro-serviços. Avaliar o impacto da nova arquitetura na latência, *throughput* e custo operacional, comparando com a versão original.

2. Granularidade dos Serviços

Comparar diferentes níveis de granularidade (serviços mais agregados vs mais finos). Analisar impacto na *performance*, *deployability* e custo de operação.

3. Monólito vs Micro-serviços: Avaliação Comparativa

Implementar e comparar as duas versões. Avaliar explicitamente ganhos e perdas em termos de *performance*, *availability*, *deployability* ou *cost*.

4. Introdução de Mecanismos de Resiliência

Após *refactoring*, introduzir mecanismos que aumentem tolerância a falhas. Avaliar impacto na *availability* e *cost*, bem como complexidade adicional no *deployment*.

Os temas são orientadores. Cada grupo deverá definir o seu objetivo de estudo e justificar a relevância arquitetural do estudo.

6. Modo de Funcionamento e Recursos

6.1 Modo de Funcionamento

O acompanhamento do projeto será realizado de forma contínua ao longo do semestre.

Semanalmente, cada grupo deverá:

- Apresentar o trabalho desenvolvido desde a última sessão;
- Identificar decisões arquiteturais tomadas;
- Explicitar dificuldades encontradas;
- Discutir próximos passos e eventuais ajustamentos ao plano experimental.

6.2 Recursos Tecnológicos

No ano letivo 2025/2026, a infraestrutura experimental disponível será baseada na *Google Cloud for Education*. Serão criadas contas institucionais para os estudantes no âmbito do programa *Google Cloud for Education*.

A plataforma poderá ser utilizada como infraestrutura de suporte à experimentação, nomeadamente para:

- Deployment de aplicações (Google Kubernetes Engine – GKE);
- Utilização de máquinas virtuais (Compute Engine);
- Bases de dados geridas (Cloud SQL);
- Monitorização e logging (Cloud Monitoring e Cloud Logging).

O foco do projeto permanece na análise arquitetural e na validação empírica, sendo a *cloud* um meio instrumental para esse efeito.

6.3 Acesso e Documentação

Os estudantes poderão consultar a documentação oficial da Google Cloud através dos seguintes recursos:

- Página principal da Google Cloud: <https://cloud.google.com/>
- Google Cloud for Education: <https://cloud.google.com/edu>
- Documentação geral: <https://cloud.google.com/docs>
- Google Kubernetes Engine (GKE): <https://cloud.google.com/kubernetes-engine>
- Cloud Monitoring: <https://cloud.google.com/monitoring>

A criação e ativação das contas Google Cloud for Education serão comunicadas pela equipa docente no início do projeto.

6.4 Bibliografia e Apoio Técnico

Durante o desenvolvimento do projeto, será disponibilizada bibliografia relevante (capítulos de livros, artigos científicos, vídeos técnicos e documentação oficial), de acordo com as necessidades identificadas em cada fase do trabalho.

A seleção de recursos será articulada com:

- tema escolhido pelo grupo;
- a decisão arquitetural analisada;
- a metodologia experimental adotada.

7. Proposta de Cronograma

Semana 2: Formação de grupos.

Semana 3: Seleção de tema; discussão e objetivos.

Semana 4-5: Definição detalhada da arquitetura e plano experimental.

Semana 6–7: Implementação mínima e configuração do ambiente experimental.

Semana 8: Revisão intermédia com demonstração funcional; **submissão relatório intermédio**.

Semana 9–13: Execução de experiências e recolha de dados.

Semana 14–15: Análise estatística e discussão arquitetural.

Semana 16-17: **Entrega final** e preparação da apresentação.

8. Avaliação

8.1 Elementos de avaliação

O projeto será avaliado tendo em conta os seguintes elementos:

Relatório: o relatório com os resultados do projeto deverá usar o seguinte template (versão *word*¹ ou *latex*²) e não deverá exceder o tamanho de 20 páginas (sem contar as figuras).

Apresentação oral dos resultados: apresentação oral dos resultados do projeto com recurso a *slides*.

¹ <https://authors.acm.org/proceedings/production-information/preparing-your-article-with-microsoft-word>, last access 14/02/2026

² <https://authors.acm.org/proceedings/production-information/preparing-your-article-with-latex>, last access 14/02/2026

Demonstração: incluída na apresentação oral, uma demonstração do funcionamento da aplicação.

A apresentação (oral + demonstração) deverá ter uma duração de aproximadamente de 30 minutos.

8.2 Critérios de avaliação

A avaliação do projeto será efetuada com base na qualidade da fundamentação arquitetural, da avaliação experimental e da coerência global do trabalho desenvolvido.

1 Fundamentação e Coerência Arquitetural

Serão avaliados:

- Clareza na identificação da decisão arquitetural analisada ou introduzida;
- Descrição explícita de pelo menos uma alternativa relevante;
- Discussão fundamentada *de trade-offs*;
- Relação clara entre a decisão arquitetural e os atributos de qualidade:
 - o *Performance*
 - o *Availability*
 - o *Deployability*
 - o *Cost*
- Coerência entre o enquadramento teórico e as opções técnicas adotadas.

Esta componente constitui o elemento central da avaliação.

2 Qualidade da Avaliação Experimental

Serão avaliados:

- Formulação clara dos objetivos e questões a analisar;
- Correta recolha e apresentação das métricas relevantes;
- Análise estatística adequada;
- Capacidade de estabelecer uma ligação explícita entre os resultados obtidos e os atributos de qualidade estudados;
- Discussão crítica dos resultados, incluindo limitações do estudo.

3 Implementação e *Deployment* (Suporte Experimental)

Serão avaliados:

- Adequação da implementação ao objetivo arquitetural;
- Justificação das configurações de recursos;
- Coerência da estratégia de deployment com os atributos analisados;
- Funcionamento estável da solução demonstrada.

A implementação é avaliada enquanto suporte à análise arquitetural e não como objetivo autónomo.

4 Discussão Crítica e Conclusões

Serão avaliados:

- Capacidade de sintetizar os resultados obtidos;
- Reflexão sobre implicações arquiteturais futuras;
- Clareza na identificação de limitações e possíveis extensões do trabalho.

5. Apresentação e Demonstração

Serão avaliados:

- Clareza da apresentação oral;
- Estrutura do relatório;
- Capacidade de responder a questões técnicas;
- Demonstração funcional da solução.

Critério	Peso
Fundamentação e coerência arquitetural	40%
Qualidade da avaliação experimental	30%
Implementação e <i>deployment</i> (suporte experimental)	15%
Discussão crítica e conclusões	10%
Apresentação oral e demonstração	5%

8.3 Auto-avaliação Individual

Cada estudante deverá submeter, em anexo ao relatório final (máximo 1 página), uma breve auto-avaliação individual indicando:

- Principais tarefas realizadas;
- Contributo técnico específico no projeto;
- Percentagem estimada de participação no trabalho do grupo.

A equipa docente poderá utilizar esta informação para ajustar a classificação individual, caso se verifiquem diferenças significativas de contributo entre membros do grupo.

9. Uso de IA

É permitida a utilização de ferramentas de Inteligência Artificial (IA) no âmbito do desenvolvimento do projeto, como por exemplo no apoio à programação, análise de dados, estruturação do relatório ou exploração preliminar de alternativas arquiteturais.

No entanto, a utilização destas ferramentas deverá obedecer a princípios de uso responsável:

- Os estudantes mantêm inteira responsabilidade pelas decisões arquiteturais adotadas, pela correção técnica do trabalho e pela validade dos resultados apresentados;
- Todas as afirmações, opções técnicas e conclusões deverão ser devidamente fundamentadas, quer por evidência empírica obtida no âmbito do projeto, quer por referência bibliográfica adequada;
- Não serão aceites afirmações não sustentadas por dados, análise experimental ou fundamentação técnica explícita;
- A utilização de IA não dispensa a compreensão dos conceitos aplicados, nem substitui a capacidade de argumentação crítica.

Sempre que possível, recomenda-se que os grupos incluam, como anexo ao relatório:

- Identificação das ferramentas de IA utilizadas;
- Exemplos de prompts relevantes;
- Breve descrição do modo como a IA contribuiu para o desenvolvimento do trabalho.

10. Data de submissão

O relatório deverá ser submetido na *Blackboard*, até ao dia 31/05, 23h59.